WEJ LAB

Support Vector Machines

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-11-04

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Explain the maximum-margin hyperplane and support vectors
- Explain the soft margin and how the cost C trades bias for variance
- Explain the kernel trick (linear, polynomial, radial kernels)
- Fit SVMs in R with `e1071` and tune them with `caret`
- Build and evaluate an SVM classifier for TB from blood gene expression

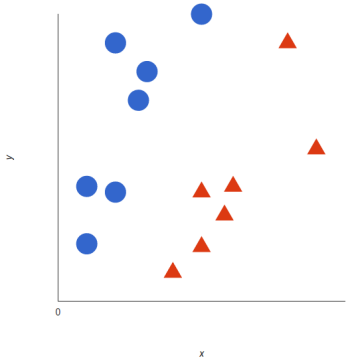
Support vector machines

- **SVMs** are supervised learning models for classification (and regression)
- Goal: find the **decision boundary** (separating **hyperplane**) that best separates two classes

Adapted from: datacamp.com/tutorial/support-vector-machines-r; *The Elements of Statistical Learning* (Hastie, Tibshirani & Friedman), Ch. 12; *An Introduction to Statistical Learning* (James et al.), Ch. 9

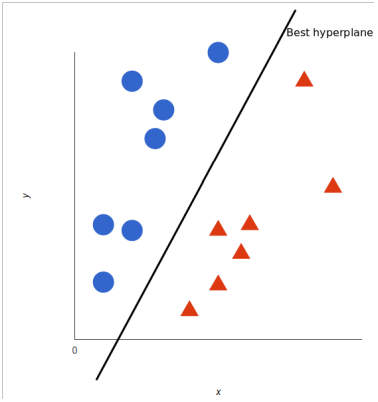
Linearly separable data

Two classes (red and blue) and two features, x and y:



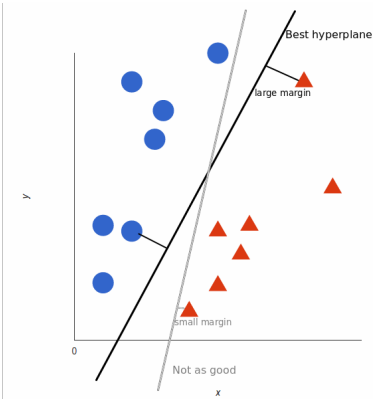
The decision boundary

An SVM finds the **hyperplane** (in two dimensions, a line) that separates the classes:



The best hyperplane: maximum margin

The best hyperplane is the one with the largest **margin**, the distance to the closest points of each class:



The maximum-margin classifier

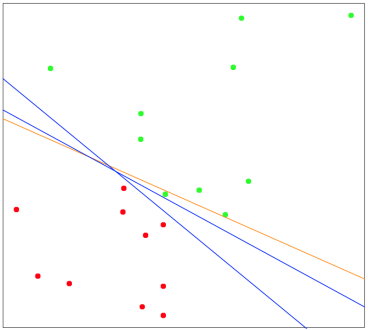


FIGURE 4.14. *A toy example with two classes separable by a hyperplane. The orange line is the least squares solution, which misclassifies one of the training points. Also shown are two blue separating hyperplanes found by the* perceptron learning algorithm *with different random starts.*

The maximum-margin classifier

For $y_i \in \{-1, 1\}$, find the hyperplane $\beta_0 + x^T\beta = 0$ that solves

$$\max_{\beta_0, \beta,\ \|\beta\|=1} M \quad \text{subject to} \quad y_i(\beta_0 + x_i^T\beta) \geq M \text{ for all } i$$

- M is the margin
- Only the points **on** the margin matter: the **support vectors**
- Predict with the sign of $\beta_0 + x^T\beta$

When classes overlap: the soft margin

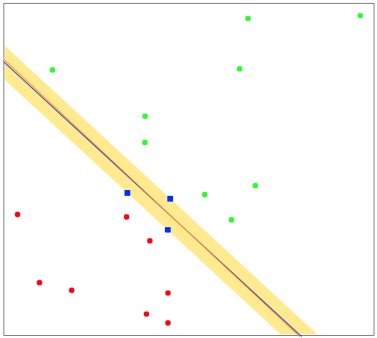


FIGURE 4.16. *The same data as in Figure 4.14. The shaded region delineates the maximum margin separating the two classes. There are three support points indicated, which lie on the boundary of the margin, and the optimal separating hyperplane (blue line) bisects the slab. Included in the figure is the boundary found using logistic regression (red line), which is very close to the optimal separating hyperplane (see Section 12.3.3).*

The soft margin and the cost C

Allow some points inside the margin or on the wrong side, with **slack** $\xi_i \geq 0$, and charge a cost for it:

$$\min_{\beta_0,\beta} \ \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{N}\xi_i \quad \text{subject to} \quad y_i(\beta_0 + x_i^T\beta) \geq 1 - \xi_i$$

- **Large** C**:** few violations allowed, narrow margin, low bias, high variance
- **Small** C**:** wide margin, more violations, higher bias, lower variance
- Tune C by cross-validation

SVM as regularized regression

The soft-margin SVM is equivalent to minimizing

$$\sum_{i=1}^{N} \max\left[0,\ 1 - y_i(\beta_0 + x_i^T\beta)\right] + \lambda\|\beta\|^2$$

- **Hinge loss:** zero for points correctly classified outside the margin
- Plus an **L2 (ridge) penalty**, with λ playing the role of $1/C$
- Same idea as the regularization lecture, with a different loss

WEJ LAB
RUTGERS HEALTH
Center for Data Science

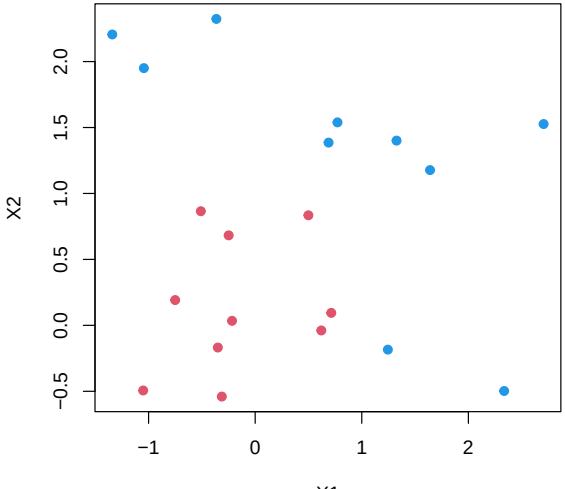
Support vector machines in R

Simulate two slightly separated classes in two dimensions:

```
set.seed(10111)
x <- matrix(rnorm(40), 20, 2)
y <- rep(c(-1, 1), c(10, 10))
x[y == 1, ] <- x[y == 1, ] + 1
plot(x, col = y + 3, pch = 19)
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Support vector machines in R



WEJ LAB

RUTGERS HEALTH
Center for Data Science

Fitting a linear SVM with e1071

`svm()` needs *y* as a factor. We set `scale = FALSE` because both features are already on the same scale; usually you **should** scale (the default).

```
dat <- data.frame(x, y = as.factor(y))
svmfit <- svm(y ~ ., data = dat, kernel = "linear", cost = 10,
              scale = FALSE)
svmfit$tot.nSV          # number of support vectors
```

```
## [1] 6
```

The 6 support vectors are the points on, inside, or beyond the margin.

Plotting the decision boundary

Predict on a grid of points to show the regions:

```
make.grid <- function(x, n = 75) {
  grange <- apply(x, 2, range)
  expand.grid(X1 = seq(grange[1, 1], grange[2, 1], length = n),
              X2 = seq(grange[1, 2], grange[2, 2], length = n))
}
xgrid <- make.grid(x)
ygrid <- predict(svmfit, xgrid)
```

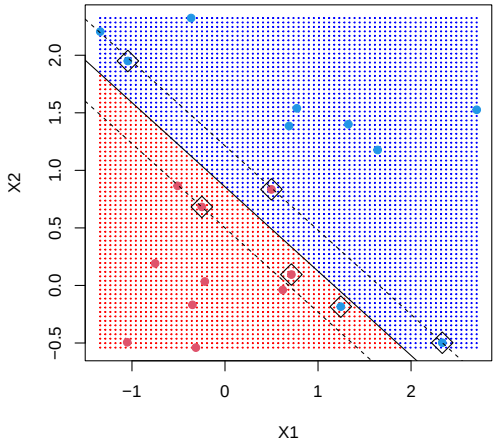
Plotting the decision boundary

The linear coefficients come from the support vectors; the boundary is $\beta_0 + x_1\beta_1 + x_2\beta_2 = 0$, and the margins are where it equals ± 1:

```
beta  <- drop(t(svmfit$coefs) %*% x[svmfit$index, ])
beta0 <- svmfit$rho
```

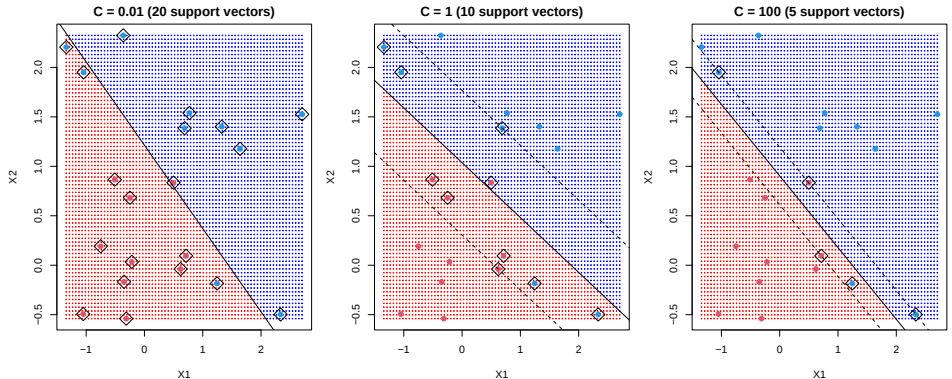
WEJ LAB
RUTGERS HEALTH
Center for Data Science

Plotting the decision boundary



Solid line: boundary. Dashed lines: margin. Diamonds: support vectors.

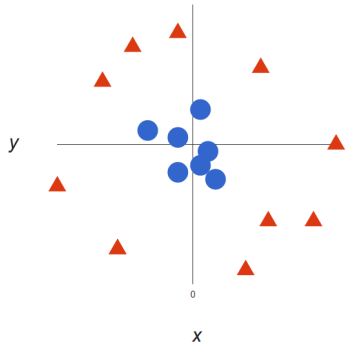
The cost C changes the margin



Small C: wide margin, many support vectors. Large C: narrow margin, few support vectors.

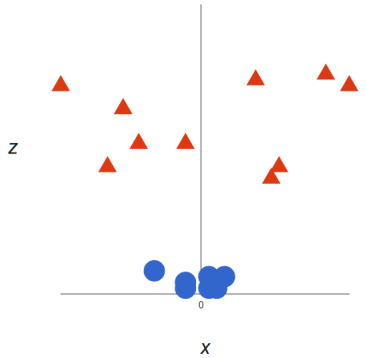
Non-linear data

Often the classes can't be separated by a line:



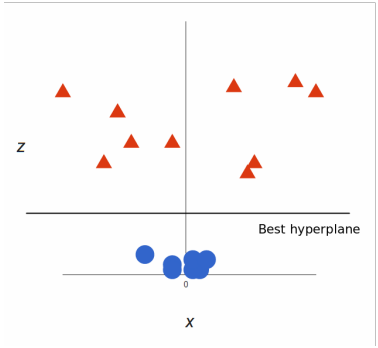
Adding a dimension

Add a third feature, $z = x^2 + y^2$ (the squared distance from the center), and plot x vs. z:



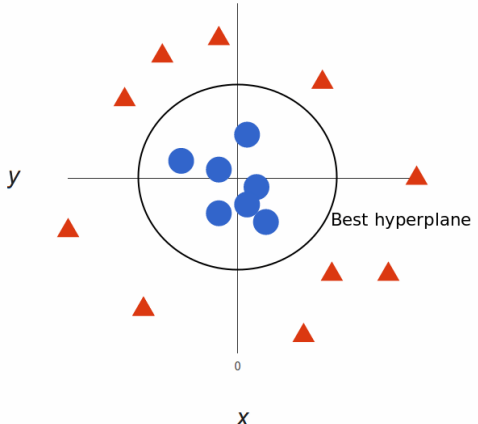
Adding a dimension

Now a flat hyperplane separates the classes:



Mapping back to two dimensions

In the original space, the boundary is a circle:



The kernel trick

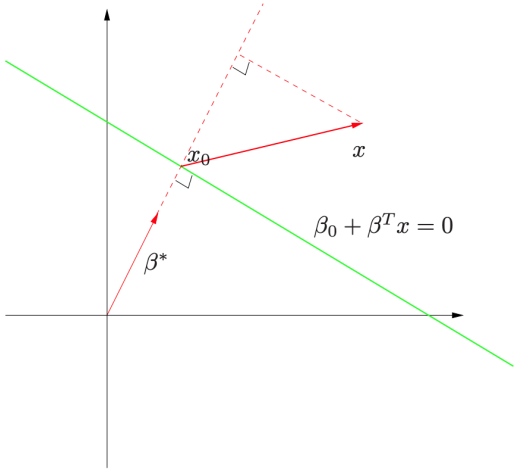
- Computing new features explicitly is expensive
- But the SVM only needs **dot products** between points, $\langle x_i, x_j \rangle$
- A **kernel** $K(x_i, x_j)$ computes the dot product in the enlarged space directly

Common kernels

Kernel	$K(x_i, x_j)$
Linear	$\langle x_i, x_j \rangle$
Polynomial	$(1 + \langle x_i, x_j \rangle)^d$
Radial (RBF)	$\exp(-\gamma \|x_i - x_j\|^2)$

Radial kernel: γ (caret calls it `sigma`) sets how local the boundary is. Large γ: wiggly boundary, risk of overfitting.

Kernels: a look at the methodology



Kernels: a look at the methodology

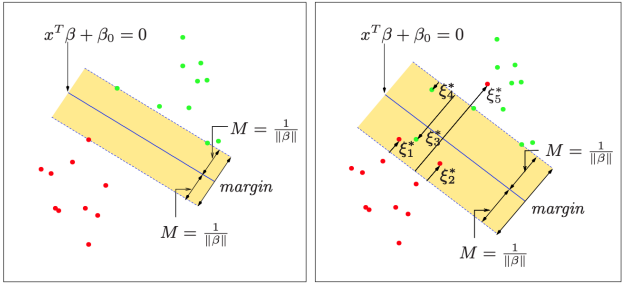


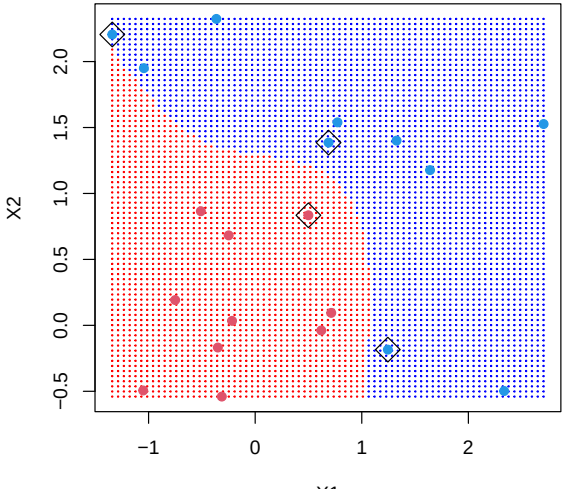
FIGURE 12.1. *Support vector classifiers. The left panel shows the separable case. The decision boundary is the solid line, while broken lines bound the shaded maximal margin of width $2M = 2/\|\beta\|$. The right panel shows the nonseparable (overlap) case. The points labeled ξ_j^* are on the wrong side of their margin by an amount $\xi_j^* = M\xi_j$; points on the correct side have $\xi_j^* = 0$. The margin is maximized subject to a total budget $\sum \xi_i \leq$ constant. Hence $\sum \xi_j^*$ is the total distance of points on the wrong side of their margin.*

Non-linear SVM: polynomial kernel

The same simulated data with a polynomial kernel:

```
svmfit_poly <- svm(y ~ ., data = dat, kernel = "polynomial",
                   cost = 10, scale = FALSE)
ygrid <- predict(svmfit_poly, xgrid)
plot(xgrid, col = c("red", "blue")[as.numeric(ygrid)],
     pch = 20, cex = .2)
points(x, col = y + 3, pch = 19)
points(x[svmfit_poly$index, ], pch = 5, cex = 2)
```

Non-linear SVM: polynomial kernel



Non-linear SVM: the ESL mixture data

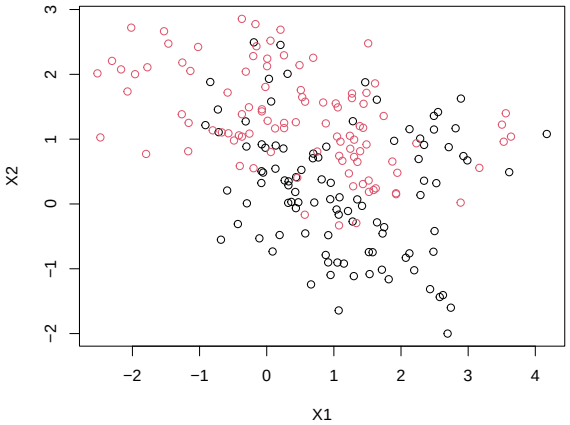
A harder example from *The Elements of Statistical Learning*: no clean separation, and the boundary must be non-linear.

```
load("ESL.mixture.rda")
str(ESL.mixture[c("x", "y", "px1", "px2")], give.attr = FALSE)
```

```
## List of 4
##  $ x  : num [1:200, 1:2] 2.5261 0.367 0.7682 0.6934 -0.0198 ...
##  $ y  : num [1:200] 0 0 0 0 0 0 0 0 0 0 ...
##  $ px1: num [1:69] -2.6 -2.5 -2.4 -2.3 -2.2 -2.1 -2 -1.9 -1.8 -1.7 ...
##  $ px2: num [1:99] -2 -1.95 -1.9 -1.85 -1.8 -1.75 -1.7 -1.65 -1.6 -1.55
```

```
dat_esl <- data.frame(y = factor(ESL.mixture$y), ESL.mixture$x)
```

Non-linear SVM: the ESL mixture data



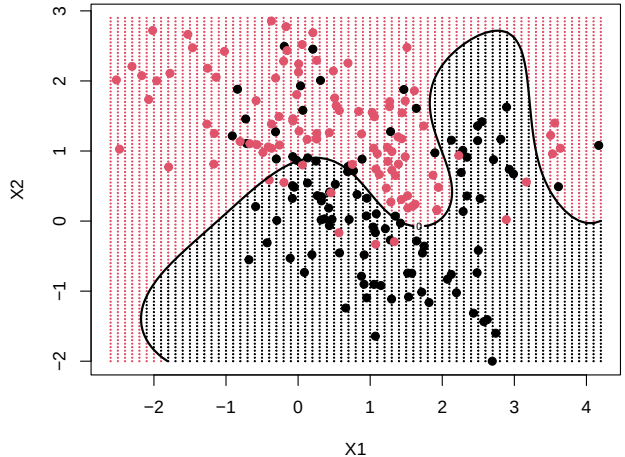
Radial-kernel SVM

Fit a radial SVM, then predict on the grid supplied with the data:

```
fit <- svm(y ~ ., data = dat_esl, scale = FALSE, kernel = "radial",
           cost = 5)
xgrid_esl <- expand.grid(X1 = ESL.mixture$px1, X2 = ESL.mixture$px2)
func <- predict(fit, xgrid_esl, decision.values = TRUE)
func <- attributes(func)$decision
fit$tot.nSV
```

```
## [1] 103
```

Radial-kernel SVM

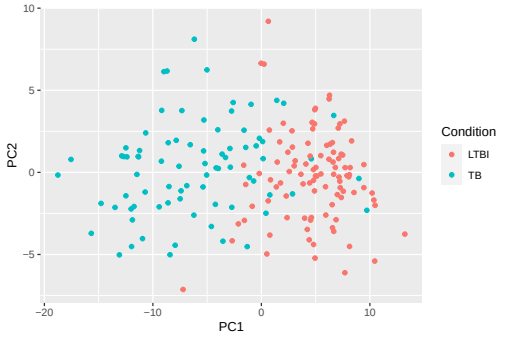


Application: TB gene expression biomarkers

Back to the TB NanoString data: 179 people with active TB or latent TB infection (LTBI), 107 blood genes.

- Can an SVM classify TB status from the full gene panel?
- Linear vs. radial kernel
- Tune C (and γ) by cross-validation; evaluate on a held-out test set

TB data: first, a look with PCA



TB and LTBI overlap in the first two PCs: we need more than two dimensions.

Tuning SVMs with caret

Scale the genes inside CV (preProcess), tune C for the linear kernel and (C, γ) for the radial kernel, choosing by AUC:

```
set.seed(1)
idx <- createDataPartition(tb$TB_Status, p = 0.7, list = FALSE)
tb_train <- tb[idx, ]; tb_test <- tb[-idx, ]
ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE,
                     summaryFunction = twoClassSummary)
set.seed(1)
svm_lin <- train(TB_Status ~ ., data = tb_train, method = "svmLinear",
                 preProcess = c("center", "scale"), metric = "ROC",
                 trControl = ctrl,
                 tuneGrid = data.frame(C = c(0.001, 0.01, 0.1, 1, 10)))
set.seed(1)
svm_rad <- train(TB_Status ~ ., data = tb_train, method = "svmRadial",
                 preProcess = c("center", "scale"), metric = "ROC",
                 trControl = ctrl, tuneLength = 8)
```

Tuning the cost C

```
svm_lin$results[, c("C", "ROC")]
```

```
##       C       ROC
## 1 1e-03 0.9628571
## 2 1e-02 0.9566667
## 3 1e-01 0.9280952
## 4 1e+00 0.9161905
## 5 1e+01 0.9161905
```

The **smallest** costs (widest margins) win: with 107 genes and 125 training samples, strong regularization helps, just as in the regularization lecture.

Test-set performance

```
library(pROC)
test_perf <- function(m) {
  p  <- predict(m, tb_test, type = "prob")[, "TB"]
  cm <- confusionMatrix(predict(m, tb_test), tb_test$TB_Status,
                        positive = "TB")
  r  <- roc(tb_test$TB_Status, p, levels = c("LTBI", "TB"),
            direction = "<", quiet = TRUE)
  c(AUC = as.numeric(auc(r)), cm$byClass[c("Sensitivity", "Specificity")]) }
```

Test-set performance

```
round(rbind(linear = test_perf(svm_lin),
            radial = test_perf(svm_rad)), 3)
```

```
##          AUC Sensitivity Specificity
## linear 0.917       0.870       0.933
## radial 0.929       0.783       0.900
```

Both kernels reach test AUC around 0.92-0.93, similar to kNN and the 3-gene score from earlier lectures. The extra flexibility of the radial kernel doesn't buy much here.

Probabilities from an SVM

- An SVM gives a **class** (which side of the boundary), not a probability
- `classProbs = TRUE` fits **Platt scaling**: a logistic regression on the SVM's decision values
- That is what let us compute ROC curves and AUC above

Strengths and weaknesses of SVMs

Strengths:

- Work well in high dimensions (e.g., many genes)
- Depend only on the support vectors
- Kernels give flexible, non-linear boundaries

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Strengths and weaknesses of SVMs

Weaknesses:

- Sensitive to the kernel, C, and γ: tune them, and scale the features
- With $p \gg N$, overfitting is easy without careful tuning
- No built-in probabilities or variable importance; slow for very large N

WEJ LAB
RUTGERS HEALTH
Center for Data Science

WEJ LAB

Summary

- SVMs find the maximum-margin hyperplane; only the support vectors matter
- The soft margin allows errors; the cost C trades bias for variance
- SVM = hinge loss + L2 penalty: regularization again
- Kernels (polynomial, radial) give non-linear boundaries without building new features
- Scale features and tune C and γ by cross-validation
- TB NanoString: linear and radial SVMs reach test AUC $\approx$ 0.92-0.93

Python companion

Python versions of these examples (`scikit-learn`'s `SVC`):

`python/week5/svm.ipynb`

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] pROC_1.18.5     e1071_1.7-14    lubridate_1.9.3 forcats_1.0.0
##  [5] stringr_1.5.1   dplyr_1.1.4     purrr_1.0.2     readr_2.1.5
##  [9] tidyr_1.3.1     tibble_3.2.1    tidyverse_2.0.0 caret_6.0-94
## [13] lattice_0.22-5  ggplot2_3.4.4
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.4         xfun_0.41            recipes_1.0.9
##  [4] tzdb_0.4.0           vctrs_0.6.5          tools_4.3.3
```